SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1741**

Register Number	**192312174**
Name	**Pavithra.S**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS:2 **Total Marks:20**

Descriptive Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behaviour and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

   Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Code of Conduct:

I _____Pavithra S___/_192312174_____ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Pavithra S

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Understanding of Concepts	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	5
Explanation	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	5
Application	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	4
Organization	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	3
Accuracy	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	2
Clarity	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	1

Solution:1 Intelligent Agents and Uninformed Search Strategies

Smart Home Automation System Using Different Types of Intelligent Agents

Introduction:

An intelligent agent is a system that observes its environment through sensors and takes actions to achieve a desired result. In a smart home automation system, intelligent agents manage lighting, temperature and security based on user behaviour and environmental conditions.

The system receives real-time inputs from motion sensors, temperature sensors, security cameras and time settings. It can also learn user preferences to improve comfort, save energy and provide safety.

Types of Intelligent Agents and Their Functions

1. Simple Reflex Agent

A simple reflex agent selects actions based only on the current percept using condition-action rules. It does not consider past experiences or maintain an internal model.

Working in a smart home:

- If motion is detected, turn ON the lights.
- If the temperature exceeds 30°C, turn ON the fan or cooling system.
- If an unauthorized door opening is detected, trigger an alarm.

Advantages:

- Simple to design and implement.
- Fast response to real-time inputs.
- Suitable for basic automation tasks.

Limitations:

- Cannot learn user preferences.
- Cannot remember previous conditions.
- May make poor decisions in complex situations.

Example: Lights turn ON whenever motion is detected, even during daytime when sufficient sunlight is available.

2. Model-Based Reflex Agent

A model-based reflex agent maintains an internal state of the environment. It uses current sensor inputs and previous information to understand changes in the home.

Working in a smart home:

- Remembers whether a room is occupied.
- Tracks the current temperature and lighting conditions.
- Uses door sensor information to estimate whether someone is at home.

- Avoids switching lights OFF immediately when a person is temporarily still.

Advantages:

- Works better in partially observable environments.
- Uses previous information for decision-making.
- Provides more reliable automation than a simple reflex agent.

Limitations:

- Requires memory and an internal model.
- Incorrect sensor data may lead to wrong decisions.
- Does not automatically learn preferences unless learning is added.

Example: If a person remains seated without movement, the system uses occupancy history to avoid turning OFF the lights unnecessarily.

3. Goal-Based Agent

A goal-based agent selects actions based on specific goals and evaluates possible actions to achieve them.

Working in a smart home:

- Maintains a comfortable room temperature.
- Ensures the house is secure when the user leaves.
- Reduces unnecessary electricity consumption.
- Plans actions based on user schedules.

Advantages:

- Makes decisions based on desired outcomes.
- Can compare different possible actions.
- Supports planning and flexible decision-making.

Limitations:

- Planning may require more computation.
- Achieving one goal may conflict with another.
- Needs clearly defined goals.

Example: Before bedtime, the agent checks room occupancy, switches OFF unnecessary lights and activates the security system while keeping essential devices running.

4. Utility-Based Agent

A utility-based agent evaluates different possible actions using a utility function and selects the action with the highest expected overall benefit.

It can balance comfort, energy efficiency, security and user preferences.

Working in a smart home:

- Selects an appropriate temperature based on user preference and energy cost.

- Adjusts lighting based on natural sunlight and occupancy.
- Balances energy saving with comfort.
- Gives high priority to security and safety.

Advantages:

- Handles conflicting goals effectively.
- Supports decision-making under uncertainty.
- Can optimize multiple performance factors.

Limitations:

- Designing a suitable utility function is difficult.
- Requires accurate data and suitable weights.
- A poorly designed utility function may prioritize the wrong outcome.

Example: If the user prefers 24°C, the agent tries to maintain that temperature while minimizing electricity use without compromising comfort or safety.

Most Effective Approach: Hybrid Intelligent Agent

A hybrid approach combining model-based, goal-based and utility-based agents with simple reflex rules is the most effective solution.

The system can function as follows:

1. Simple reflex rules: Respond immediately to critical events, such as smoke detection or security alerts.
2. Model-based component: Maintains information about occupancy, temperature, time and device status.
3. Goal-based component: Plans actions to achieve comfort, safety and energy-saving goals.
4. Utility-based component: Chooses the best action among available alternatives.
5. Learning component: Learns user preferences over time, such as preferred temperature and lighting levels.

Security rules should override energy-saving decisions. For example, the system must not disable an alarm simply to reduce power consumption.

Conclusion

A hybrid intelligent agent provides the best solution for smart home automation because it combines fast responses, memory, planning, optimization and learning. This approach improves user comfort, energy efficiency and home security while adapting to changing environmental conditions and user behaviour.

2. Robot Maze Solving Using Uniform Cost Search (UCS) and Iterative Deepening Search (IDS)

Introduction

A robot placed in an unknown maze must find a path from its starting position to the exit without knowing the complete maze layout.

The robot uses sensors to detect walls, open paths and possible movements. Uninformed search algorithms help it explore possible paths without using heuristic information about the distance to the exit.

The two important uninformed search strategies are Uniform Cost Search (UCS) and Iterative Deepening Search (IDS).

Problem Formulation

The maze problem can be represented using the following elements:

- Initial state: Robot's starting position.
- State space: All reachable positions in the maze.
- Actions: Move up, down, left or right.
- Transition model: Describes the new position after a valid movement.
- Goal test: Checks whether the robot has reached the exit.
- Path cost: Total cost of movements from the start to the exit.

The robot explores the maze, identifies possible routes and selects a suitable path to reach the goal.

Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the lowest accumulated path cost.

It uses a priority queue to store unexplored nodes based on their total cost from the starting position.

Working Procedure

1. Insert the starting position into the priority queue with cost 0.
2. Remove the position with the lowest path cost.
3. Check whether it is the goal.
4. If it is not the goal, explore its valid neighbouring positions.
5. Calculate the cost of reaching each neighbour.
6. Update the priority queue with better paths.
7. Repeat until the exit is reached or no path exists.

Example

Suppose the robot has two possible paths to the exit:

- Path A: 8 steps, with total cost 8.

- Path B: 5 steps, with total cost 10 due to difficult terrain.

UCS selects Path A because its total cost is lower.

Thus, UCS finds the least-cost path when step costs are positive and the algorithm is implemented correctly.

Advantages

- Complete when the branching factor is finite and step costs are bounded below by a positive value.
- Finds an optimal least-cost path under these conditions.
- Suitable for mazes with different movement costs.

Disadvantages

- May explore many unnecessary paths.
- Requires considerable memory for the frontier and explored states.
- Can be slow in large mazes.

Iterative Deepening Search (IDS)

Iterative Deepening Search repeatedly performs depth-limited depth-first search, increasing the depth limit each time until the goal is found.

Working Procedure

1. Start with depth limit 0.
2. Search the maze up to the current depth limit.
3. If the exit is not found, increase the limit by 1.
4. Restart the search from the starting position.
5. Continue until the goal is found.

Example

- Depth 0: Check the starting position.
- Depth 1: Explore positions one move away.
- Depth 2: Explore positions up to two moves away.
- Depth 3: Explore positions up to three moves away.
- Continue until the exit is found.

IDS is useful when the depth of the solution is unknown.

Advantages

- Requires less memory than UCS.
- Complete for finite branching factor.
- Finds a shallowest solution when all actions have equal cost.
- Does not require prior knowledge of the maze depth.

Disadvantages

- Repeats the exploration of nodes in earlier iterations.
- May waste time restarting the search.
- Does not guarantee the least-cost path when movement costs differ.

Time and Space Complexity

Let:

- (b) = branching factor (average number of successors)
- (d) = depth of the shallowest solution
- (C^*) = optimal solution cost
- (epsilon) = minimum positive step cost

Explanation:

UCS: Time and memory can grow exponentially because many frontier nodes may need to be stored. Its precise complexity depends on path costs and the state graph.

IDS: Time is exponential in the solution depth, but memory is much lower because it performs depth-first searches repeatedly.

For IDS, \(O(bd)\) is the usual tree-search space bound, excluding additional implementation-specific memory.

For UCS, in a tree-search formulation, a common space bound is \(O(b^{1+\lfloor C^*/\epsilon \rfloor})\). Graph-search implementations also store explored states.

Relationship with Intelligent Agents

Different intelligent agents affect how the robot explores and solves the maze.

1. Simple Reflex Agent

- Makes decisions based on the current sensor input.
- May turn left when the left path is open.
- Cannot reliably remember visited locations.
- Can get trapped in loops.

Application: Suitable only for simple maze-following rules.

2. Model-Based Agent

- Maintains an internal map of explored cells.
- Remembers walls, open paths and visited positions.
- Uses previous observations to avoid unnecessary exploration.

Application: Highly useful because the maze is initially unknown and the robot must gradually build its map.

3. Goal-Based Agent

- Has the goal of reaching the maze exit.

- Evaluates possible paths to the exit.
- Uses search algorithms to plan movements.

  Application: Useful for selecting a route and achieving the destination.

  4. Utility-Based Agent
- Evaluates paths based on distance, energy, time and safety.
- Selects a route with the highest overall benefit or lowest total cost.
- Can consider risks such as slippery surfaces or narrow passages.

  Application: Useful when the robot must balance travel time, battery usage and safety.

Most Effective Combination and Justification

The most effective approach is a model-based, goal-based robot using UCS or IDS according to the maze conditions, with utility-based route evaluation when additional costs matter.

Justification

- Model-based agent: Necessary to maintain knowledge of the partially observed maze.
- Goal-based agent: Ensures that the robot plans specifically toward the exit.
- UCS: Best when movement costs differ and finding the least-cost route is important.
- IDS: Best when movement costs are equal and memory is limited.
- Utility-based evaluation: Helpful when energy, safety and time must be considered together.

For an unknown maze, the robot cannot plan the complete route immediately. It must explore, update its internal map and replan whenever it discovers new information. Search algorithms can be applied to the known portion of the map, with additional exploration as needed.

Conclusion

A model-based and goal-based intelligent agent combined with UCS or IDS provides an effective maze-solving solution. UCS is preferred for different movement costs, while IDS is suitable for equal-cost movements and memory-limited robots. Adding utility-based decision-making improves route selection when energy consumption, travel time and safety are important.